

[12] Qdrant: High-Performance Vector Search at Scale

개발사: Qdrant (오픈소스)

유형: 전문 벡터 검색 엔진, Rust 기반

출시년도: 2024년 주요 기술 문서 기반

역할군: (C) 경쟁/비교 시스템

요약

Qdrant는 Milvus와 함께 **전문 벡터 DB의 양대 산맥**으로, 필터링된 벡터 검색을 특히 잘 지원한다. Rust로 작성되어 메모리 안전성과 뛰어난 성능을 동시에 제공하며, **gRPC/REST API**를 통해 다양한 클라이언트와 통합할 수 있다.

Qdrant의 핵심 특징:

1. **Rust 기반 고성능:** GC(가비지 컬렉터) 없는 예측 가능한 지연 시간
2. **HNSW 기반 + 페이로드 필터링:** 벡터 검색과 메타데이터 필터링을 동시에 수행
3. **지능형 필터 전략 선택:** 선택도에 따라 자동으로 필터 우선/벡터 우선 전략 전환
4. **양자화 기법:** Scalar, Product, Binary Quantization으로 메모리 효율성과 성능의 밸런스 조정
5. **분산 배포:** Raft 기반 샤딩과 복제로 대규모 시스템 구축 가능

Qdrant도 Milvus와 마찬가지로 단일 컬렉션 내 필터링에 집중하며, **멀티 테이블 조인 쿼리는 지원하지 않는다**. 이것이 범용 벡터 DB + Exqutor 스타일의 최적화가 필요한 이유를 보여준다.

상세분석

12.1 핵심 특징: Rust의 장점 극대화

Rust 기반 고성능의 의미:

Rust는 C/C++처럼 메모리를 직접 관리하면서도, 컴파일 타임에 메모리 안전성을 검증한다. 이를 통해:

- **메모리 안전성**: 버퍼 오버플로우, use-after-free 같은 메모리 오류가 원천 차단된다.
- **제로 코스트 추상화**: 추상화 계층이 있어도 컴파일러가 최적화하면 C/C++와 같은 성능을 낸다.
- **GC 없음**: 자동 메모리 회수(가비지 컬렉션)가 없으므로, 예측 불가능한 지연 시간(GC pause)이 발생하지 않는다. 이는 벡터 검색같이 **일정한 응답 시간이 중요한 시스템**에 매우 중요하다.
- **멀티스레드 안전성**: 소유권 시스템 덕분에 데이터 경쟁(data race)이 컴파일 타임에 감지된다.

실제 성능 비교:

- Java 기반 Elasticsearch보다 **3~5배** 빠른 색인 구축
- Python 기반 시스템 대비 **10배 이상** 높은 처리량
- 메모리 사용량 **50% 이하** (Milvus 대비)

12.2 HNSW 기반 + 페이로드 필터링: 동시적 처리

HNSW 그래프 탐색과 필터링의 동시성:

Qdrant의 가장 핵심적인 기술적 기여는 벡터 유사도 검색(HNSW)과 메타데이터 필터링을 **동시에** 수행하는 구조이다.

```
# Qdrant 검색 예시
client.search(
    collection_name="products",
    query_vector=[0.1, 0.2, ...],
    query_filter=Filter(
        must=[FieldCondition(key="price", range=Range(lt=1000))]
    ),
    limit=10
)
```

필터 선택도에 따른 자동 전략 전환:

Qdrant의 핵심 통찰은 필터의 선택도(어느 정도 비율의 데이터가 필터를 통과하는가)에 따라 최적의 검색 전략이 달라진다는 것이다.

1. **선택도 높음** (결과 많음, 예: 80%의 데이터가 조건을 만족):
2. HNSW 그래프를 탐색하면서 조건을 불만족하는 노드를 건너뛴
3. 거의 모든 데이터가 조건을 만족하므로, 필터를 먼저 적용하는 것이 비효율적
4. HNSW 탐색 중 조건 체크: O(탐색 노드 수)
5. **선택도 낮음** (결과 적음, 예: 5%의 데이터만 조건을 만족):
6. 먼저 메타데이터 인덱스로 조건을 만족하는 데이터를 추린다.
7. 남은 작은 데이터셋에 대해 벡터 검색을 수행한다.
8. 브루트포스 + 필터: O(필터링 후 크기)

이렇게 자동으로 전환하는 것이 Qdrant의 장점이다. 하지만 이 전환 결정에 사용되는 카디널리티 추정(필터링 후 몇 개가 남을 것인가)이 정확해야 한다.

12.3 양자화 기법: 메모리-성능 트레이드오프

Qdrant는 3가지 양자화 기법을 제공하여 메모리와 성능의 밸런스를 조정할 수 있다:

Scalar Quantization (스칼라 양자화):

- 각 벡터 컴포넌트를 32비트 부동소수점에서 8비트 정수로 변환
- 메모리 사용량: **1/4로 감소** (768차원 벡터가 768바이트로 줄어듦)
- 정확도 손실: 약 2~5% (작은 손실)
- 적합한 경우: 메모리가 부족하지만 높은 정확도가 필요한 경우

Product Quantization (곱셈 양자화):

- 벡터를 여러 서브벡터로 분할하고, 각 서브벡터를 독립적으로 양자화
- 예: 768차원을 32개의 24차원 서브벡터로 분할
- 메모리 사용량: **1/32로 감소**
- 정확도: 스칼라 양자화보다 좀 더 손실 (하지만 여전히 합리적인 수준)
- 적합한 경우: 극도로 메모리가 제한적인 환경 (모바일 기기, 엣지 컴퓨팅)

Binary Quantization (이진 양자화):

- 각 컴포넌트를 1비트(0 또는 1)로 변환하는 극단적 압축
- 메모리 사용량: **1/256로 감소**
- 정확도: 상당한 손실 (재현율 70~80%)
- 적합한 경우: 초고속 검색이 필요하고 정확도를 어느 정도 포기할 수 있는 경우 (실시간 대규모 검색)

12.4 분산 배포: Raft 기반 일관성 보장

Qdrant는 분산 환경에서 대규모 데이터를 관리할 수 있도록 설계되었다.

샤딩(Sharding):

- 벡터를 여러 샤드로 분할하여 여러 노드에 분산 저장
- 각 샤드는 부분 인덱스를 가짐
- 검색 시 모든 샤드의 부분 결과를 병합하여 전체 결과를 반환

복제(Replication):

- 각 샤드의 복제본을 다른 노드에 저장하여 가용성 확보
- 노드 장애 시에도 복제본에서 데이터를 계속 제공

Raft 합의 프로토콜:

- 분산 환경에서 여러 노드가 데이터 상태를 일관되게 유지하기 위해 Raft 프로토콜 사용
- 쓰기 작업 시 대다수(quorum) 노드가 승인하면 성공으로 간주
- 강한 일관성(Strong Consistency) 보장: 모든 노드가 동일한 상태

12.5 API 인터페이스: gRPC와 REST

Qdrant는 두 가지 API를 지원하여 다양한 클라이언트 환경에 대응한다.

gRPC API:

- 고성능 이진 프로토콜 기반
- 네트워크 대역폭이 제한적인 환경에 효율적
- 스트리밍 쿼리 지원

REST API:

- HTTP 기반으로 모든 표준 HTTP 클라이언트에서 접근 가능
- 디버깅과 테스트가 용이 (curl 등으로 직접 쿼리 가능)
- 웹 브라우저에서도 직접 접근 가능

12.6 본 논문과의 관계

Qdrant는 Milvus와 마찬가지로 **단일 컬렉션 내 메타데이터 필터링**에 특화되어 있다. Qdrant의 자동 필터 전략 전환 기능(선택도 기반)은 흥미로운 접근이지만, 이 전환 결정을 위한 카디널리티 추정이 어떤 수준인지는 공개되지 않았다.

SQL의 조인·서브쿼리·집계는 불가능하므로, **멀티 테이블 VAQ에는 여전히 범용 벡터 DB + Exqutor 스타일의 최적화가 필요**하다는 것을 보여주는 또 다른 사례이다.

만약 Qdrant에 Exqutor의 ECQO를 적용한다면, 필터 선택도를 더욱 정확히 추정할 수 있어 자동 전략 선택의 정확성이 향상될 것이다. 하지만 Qdrant는 SQL을 지원하지 않으므로 직접 적용은 불가능하고, 개념적으로만 유사한 문제를 다루고 있다.

추가 제기 문제

Qdrant의 필터 선택도 기반 자동 전략 전환은 정교한 휴리스틱일 수 있지만, 선택도를 부정확하게 추정하면 오히려 성능이 저하될 수 있다. 예를 들어, 실제 선택도가 10%인데 50%로 추정되면, 높은 선택도로 판단하여 HNSW 그래프를 먼저 탐색하게 되고, 이는 불필요한 벡터 계산을 야기한다.

또한 Qdrant의 분산 환경에서 여러 샤드의 부분 결과를 병합할 때, 각 샤드가 반환한 top-k가 전체 top-k를 정확히 근사하는지에 대한 이론적 분석이 부족해 보인다. 이는 향후 연구 주제로 남아있다.